

LAPORAN PRAKTIKUM WORKSHOP WEB DASAR



OLEH:

Nama	:Zafir Amal Maulana
Nim	:2025573010141
Kelas	: TI 1\C
Jurusan	: Teknologi Informasi dan Komputer
Dosen	: M. Reza Zulman S,ST.M.Sc

**PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE
2026**

LEMBAR PENGESAHAN

Nama Mata Kuliah	: Workshop Web Dasar
Judul Pratikum	: Praktikum Javascript DOM
Hari/Tanggal Pratikum	: 2 Juni 2026
Tanggal Penyerahan	: 8 Juni 2026
Tempat	: Laboratorium (TIK.105)
Nama Mahasiswa	: Zafir Amal Maulana
NIM	: 2025573010141
Program Studi/Kelas	: Teknik Informatika / TI.1C

Mengetahui
Dosen Pengajar,

M. Reza Zulman S,ST.M.Sc

DAFTAR ISI

LEMBAR PENGESAHAN	i
KATA PENGANTAR	iii
BAB I	1
PENDAHULUAN	1
1.1 Latar belakang	1
1.2 TujuanPraktikum	2
1.3 Manfaat Praktikum	2
BAB II LANDASAN TEORI	3
2.1 Javascript	3
2.2 Document Obejct Model (DOM)	3
2.3 Manipulasi Elemen DOM.....	3
2.4 Mengubah Konten dan Atribut HTML	4
2.5 Manipulasi CSS dengan JavaScript.....	4
2.6 Manfaat JavaScript DOM dalam Pengembangan Web	4
BAB III	5
METODOLOGI PRAKTIKUM	5
3.1 Alat Dan Bahan	5
3.2 Langkah-Langkah.....	5
BAB IV	40
PENUTUP	40
4.1 KESIMPULAN	40
4.2 SARAN	41

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum yang berjudul “Praktikum JavaScript DOM” dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban penulis dalam melaksanakan kegiatan praktikum serta untuk memenuhi tugas yang telah diberikan.

Laporan ini bertujuan untuk mempelajari dan memahami penggunaan JavaScript DOM (Document Object Model) dalam pengembangan halaman web. Melalui praktikum ini, penulis dapat mengetahui cara memanipulasi elemen HTML secara dinamis menggunakan JavaScript, sehingga halaman web dapat menjadi lebih interaktif dan responsif terhadap tindakan pengguna.

Selain itu, praktikum ini juga memberikan pemahaman mengenai konsep-konsep dasar DOM, seperti memilih elemen HTML, mengubah isi dan atribut elemen, memodifikasi gaya (style), serta menangani berbagai peristiwa (event) yang terjadi pada halaman web. Penulis juga mempelajari bagaimana JavaScript dapat digunakan untuk menciptakan interaksi yang lebih menarik dan meningkatkan pengalaman pengguna saat mengakses website.

Praktikum ini membantu penulis memahami penerapan JavaScript dalam pengembangan web modern, khususnya dalam menghubungkan struktur HTML dengan logika pemrograman sehingga halaman web dapat merespons input pengguna secara langsung. Dengan memanfaatkan DOM, pengembang dapat mengelola dan memperbarui konten halaman tanpa perlu memuat ulang halaman secara keseluruhan. Melalui praktikum ini, penulis memperoleh tambahan wawasan dan keterampilan dalam bidang pengembangan web, khususnya dalam penggunaan JavaScript DOM untuk membangun halaman web yang lebih dinamis, interaktif, dan mudah digunakan. Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan baik dari segi penulisan maupun isi materi yang disampaikan. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari berbagai pihak demi perbaikan dan penyempurnaan laporan ini di masa yang akan datang.

Akhir kata, penulis berharap semoga laporan ini dapat memberikan manfaat, menambah wawasan, serta menjadi referensi bagi pembaca yang ingin mempelajari lebih lanjut mengenai JavaScript DOM dan penerapannya dalam pengembangan halaman web yang interaktif.

BAB I

PENDAHULUAN

1.1 Latar belakang

Perkembangan teknologi informasi dan internet saat ini semakin pesat dan memberikan pengaruh besar dalam berbagai bidang kehidupan, terutama dalam penyebaran informasi secara digital. Salah satu media yang banyak digunakan untuk menyampaikan informasi adalah website. Website dapat diakses oleh banyak orang melalui internet dan dimanfaatkan untuk berbagai keperluan seperti pendidikan, bisnis, hiburan, dan komunikasi.

Dalam pembuatan sebuah website, tidak hanya tampilan yang menarik yang diperlukan, tetapi juga kemampuan website untuk berinteraksi dengan pengguna secara dinamis. Untuk mewujudkan hal tersebut, digunakan JavaScript sebagai bahasa pemrograman yang berfungsi untuk menambahkan interaktivitas pada halaman web. Salah satu konsep penting dalam JavaScript adalah Document Object Model (DOM), yaitu representasi struktur dokumen HTML yang memungkinkan pengembang untuk mengakses, mengubah, menambah, maupun menghapus elemen pada halaman web secara langsung.

Pemahaman mengenai JavaScript DOM sangat penting bagi seseorang yang ingin mempelajari pengembangan web modern. Dengan DOM, pengembang dapat membuat halaman web yang lebih responsif terhadap tindakan pengguna, seperti mengubah konten secara otomatis, memvalidasi formulir, menampilkan pesan interaktif, serta menangani berbagai peristiwa (event) tanpa harus memuat ulang halaman.

Melalui praktikum JavaScript DOM, mahasiswa atau siswa dapat mempelajari bagaimana cara memanipulasi elemen HTML menggunakan JavaScript, mengelola event, serta menciptakan interaksi yang menarik pada halaman web. Praktikum ini bertujuan agar peserta dapat memahami konsep dasar hingga penerapan DOM dalam pengembangan website yang dinamis dan interaktif.

Dengan mempelajari JavaScript DOM secara mendalam, diharapkan peserta mampu memahami hubungan antara HTML, CSS, dan JavaScript dalam membangun sebuah website. Selain itu, peserta juga diharapkan memiliki keterampilan dalam mengembangkan halaman web yang lebih interaktif, responsif, dan sesuai dengan kebutuhan pengguna serta perkembangan teknologi web saat ini.

1.2 TujuanPraktikum

1. Memahami apa itu DOM (Document Object Model) dan hubungannya dengan HTML
2. Memilih elemen dengan `querySelector`, `querySelectorAll`, `getElementById`
3. Memanipulasi elemen: `innerHTML`, `textContent`, `classList`, `style`, `setAttribute`
4. Menangani event dengan `addEventListener` — `click`, `input`, `submit`, `keydown`
5. Membuat, menambah, dan menghapus elemen DOM secara dinamis
6. Mengimplementasikan form validation real-time tanpa library

1.3 Manfaat Praktikum

1. Menambah pemahaman mengenai hubungan antara HTML, CSS, dan JavaScript dalam pengembangan website.
2. Meningkatkan keterampilan dalam memanipulasi elemen halaman web melalui DOM.
3. Membantu memahami cara membuat interaksi pengguna yang lebih menarik dan dinamis pada website.
4. Memberikan pengalaman praktis dalam penggunaan JavaScript untuk pengembangan web modern.
5. Menjadi dasar untuk mempelajari teknologi web yang lebih lanjut, seperti framework JavaScript dan pengembangan aplikasi web interaktif.

BAB II

LANDASAN TEORI

2.1 Javascript

JavaScript adalah bahasa pemrograman yang digunakan untuk membuat halaman web menjadi lebih interaktif dan dinamis. JavaScript bekerja di sisi klien (client-side) dan dapat dijalankan langsung oleh browser tanpa memerlukan proses kompilasi terlebih dahulu. Dengan JavaScript, pengembang dapat mengelola data, memproses input pengguna, membuat animasi, serta memanipulasi elemen-elemen pada halaman web. JavaScript merupakan salah satu teknologi utama dalam pengembangan web selain HTML dan CSS. HTML berfungsi sebagai struktur halaman, CSS digunakan untuk mengatur tampilan, sedangkan JavaScript digunakan untuk memberikan fungsi dan interaktivitas pada halaman web.

2.2 Document Object Model (DOM)

Document Object Model (DOM) adalah representasi dokumen HTML dalam bentuk struktur pohon (tree structure) yang terdiri dari berbagai node atau objek. DOM memungkinkan JavaScript untuk mengakses, membaca, dan memodifikasi isi maupun struktur halaman web secara dinamis. Melalui DOM, setiap elemen HTML seperti heading, paragraf, tombol, dan form dapat diakses serta dimanipulasi menggunakan JavaScript. Dengan demikian, perubahan pada halaman web dapat dilakukan tanpa harus memuat ulang halaman (reload).

2.3 Manipulasi Elemen DOM

Manipulasi DOM merupakan proses mengubah isi, atribut, maupun gaya suatu elemen HTML menggunakan JavaScript. Beberapa metode yang sering digunakan antara lain:

- `getElementById()`
- `getElementsByClassName()`
- `getElementsByTagName()`
- `querySelector()`
- `querySelectorAll()`

Metode-metode tersebut digunakan untuk memilih elemen HTML yang akan dimanipulasi sesuai kebutuhan program.

2.4 Mengubah Konten dan Atribut HTML

JavaScript memungkinkan pengembang untuk mengubah isi dan atribut elemen HTML secara dinamis. Beberapa properti yang sering digunakan adalah:

- `innerHTML`
- `textContent`
- `setAttribute()`
- `getAttribute()`

Dengan properti dan metode tersebut, konten halaman dapat diperbarui sesuai dengan interaksi pengguna.

2.5 Manipulasi CSS dengan JavaScript

Selain mengubah isi halaman, JavaScript juga dapat digunakan untuk mengubah tampilan elemen melalui DOM. Perubahan dapat dilakukan pada warna, ukuran, posisi, maupun gaya lainnya menggunakan properti **style**. Contohnya adalah mengubah warna latar belakang, ukuran teks, atau menampilkan dan menyembunyikan elemen tertentu sesuai kebutuhan aplikasi.

2.6 Manfaat JavaScript DOM dalam Pengembangan Web

Penggunaan JavaScript DOM memberikan berbagai manfaat dalam pengembangan website, antara lain:

1. Membuat halaman web lebih interaktif.
2. Memungkinkan perubahan konten secara dinamis tanpa reload halaman.
3. Meningkatkan pengalaman pengguna (User Experience).
4. Mempermudah pengelolaan elemen HTML dan CSS.
5. Mendukung pembuatan aplikasi web modern yang responsif dan efisien.

Dengan memahami konsep JavaScript DOM, pengembang dapat membangun website yang tidak hanya informatif tetapi juga mampu memberikan interaksi yang baik kepada pengguna.

BAB III

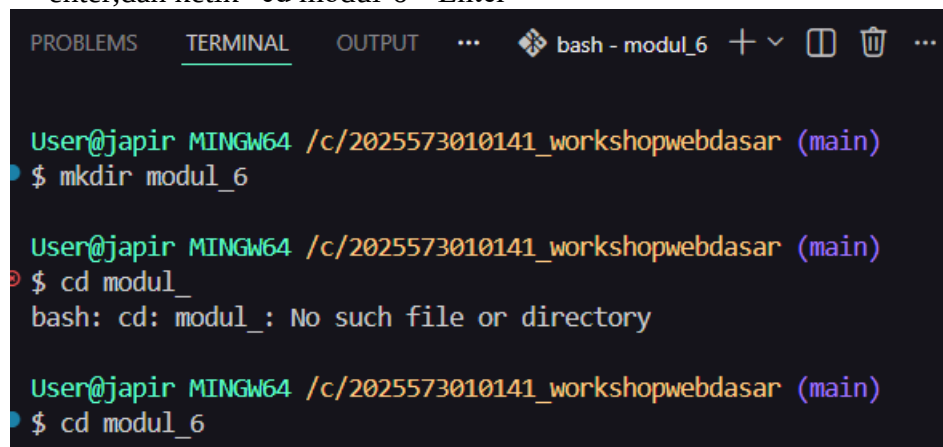
METODOLOGI PRAKTIKUM

3.1 Alat Dan Bahan

- PC/Laptop
- Koneksi internet(opsional)
- VScode
- Git (untuk version control)
- Github (untuk menyimpan project secara online)

3.2 Langkah-Langkah

1. pertama Membuat folder baru dan file baru
buka Gitbash Kemudian navigasikan ke folder repository GitHub kamu yang sudah di-clone.buat folder baru lalu ketik “mkdir modul-6 ”,lalu enter,dan ketik “cd modul-6” Enter



```
PROBLEMS  TERMINAL  OUTPUT  ...  bash - modul_6  +  -  [ ]  [ ]  ...

User@japir MINGW64 /c/2025573010141_workshopwebdasar (main)
$ mkdir modul_6

User@japir MINGW64 /c/2025573010141_workshopwebdasar (main)
$ cd modul_
bash: cd: modul_: No such file or directory

User@japir MINGW64 /c/2025573010141_workshopwebdasar (main)
$ cd modul_6
```

(pastikan kamu berada di folder **modul-6**) lalu ketik prompt berikut di gitbash untuk membuat file baru di dalam modul-6.

“touch index. dan html script.js”

“touch manipulasi. dan html manipulasi.js”

“touch create-delete.html dan create-delete.js”

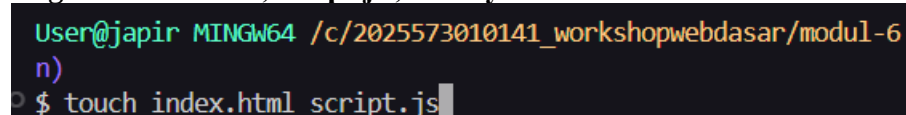
“touch events.html dan events.js”

“touch form-validasi.html, form-validasi.css, dan form-validasi.js ”

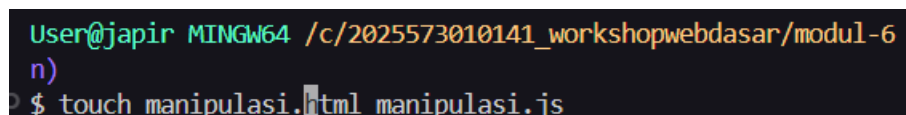
“touch latihan-1.html, dan latihan-1.js”

“touch latihan-2.html, dan latihan-2.js”

Folder tugas “index.html, script.js ,dan style.css”



```
User@japir MINGW64 /c/2025573010141_workshopwebdasar/modul-6
(n)
$ touch index.html script.js
```



```
User@japir MINGW64 /c/2025573010141_workshopwebdasar/modul-6
(n)
$ touch manipulasi.html manipulasi.js
```

```
User@japir MINGW64 /c/2025573010141_workshopwebdasar/modul-6  
n)  
$ touch create-delete.html create-delete.js
```

```
User@japir MINGW64 /c/2025573010141_workshopwebdasar/modul-6  
n)  
$ touch events.html events.js
```

```
User@japir MINGW64 /c/2025573010141_workshopwebdasar/modul-6 (mai  
n)  
$ touch form-validasi.html form-validasi.css form-validasi.js
```

```
User@japir MINGW64 /c/2025573010141_workshopwebdasar/modul-6  
n)  
$ touch latihan-1.html latihan-1.js
```

```
User@japir MINGW64 /c/2025573010141_workshopwebdasar/modul-6  
n)  
$ touch latihan-2.html, dan latihan-2.js
```

Setelah membuat file” tersebut ketikkan kode” berikut

A. Bagian -1 persiapan modul-6

klik file index.html masukkan code berikut.simpan file dan commit ke github

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>JavaScript DOM</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <!-- Konten akan kita tambahkan di setiap bagian -->

  <!-- Script di akhir body: DOM sudah siap sebelum JS dijalankan -->
  <script src="script.js"></script>
</body>
</html>
```

```
# Pastikan anda berada pada root folder "nama-repository". Jika tidak, keluar dari modul 6 dengan perintah
cd ..

# Tambahkan semua file ke git
git add .

# Commit code and
git commit -m "bagian-1-persiapan modul 6"
```

DOM adalah representasi dokumen HTML sebagai pohon objek yang bisa dimanipulasi JavaScript.

Ketika browser membaca file HTML, ia membangun struktur pohon di memori:

document

```
├── html
│   ├── head
│   │   ├── title
│   │   └── link
│   └── body
│       ├── header
│       │   └── h1 → 'Judul Halaman'
│       ├── main
│       │   ├── p → 'Paragraf 1'
│       │   └── ul
│       │       ├── li → 'Item 1'
│       │       └── li → 'Item 2'
│       └── footer
```

Setiap kotak dalam pohon disebut Node. Ada beberapa jenis node:

- Element node — elemen HTML (<div>, <p>, <button>)

- Text node — teks di dalam elemen

- Attribute node — atribut elemen (class, id, href)

JavaScript mengakses DOM melalui objek global `document`. Dari document, kita bisa menavigasi

dan memanipulasi seluruh halaman.

Cara Menemukan Elemen di DOM

Ada beberapa cara memilih elemen. Yang paling modern dan fleksibel:

- `document.querySelector(selector)` — mengembalikan satu elemen pertama yang cocok.

Selector-nya persis seperti CSS: '#id', '.class', 'p', 'form input[type=email]'.

- `document.querySelectorAll(selector)` — mengembalikan semua elemen yang cocok

sebagai `NodeList`. `NodeList` mirip array, tapi butuh `Array.from()` atau spread untuk pakai `.map()` dll.

- `document.getElementById('id')` — lebih cepat tapi hanya untuk ID. Cara lama tapi masih sering dipakai.

Cara lama yang masih ada di codebase orang lain:

- `document.getElementsByClassName('nama')` — mengembalikan `HTMLCollection` (bukan array)

- `document.getElementsByTagName('p')` — semua `<p>`

Navigasi relatif — jika sudah punya satu elemen:

- `parentElement` — elemen parent

- `children` — semua anak langsung (`HTMLCollection`)

- `firstElementChild` / `.lastElementChild` — anak pertama/terakhir

- `nextElementSibling` / `.previousElementSibling` — saudara sebelum/sesudah

- `closest(selector)` — cari ancestor terdekat yang cocok (sangat berguna!)

B. Bagian -2 DOM tree

selanjutnya update pada bagian <body> pada file index.html dengan code berikut.

```
1 <!doctype html>
2 <html lang="id">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <title>Document</title>
7     <link rel="stylesheet" href="style.css" />
8   </head>
9   <body>
10    <div id="app">
11      <h1 class="judul">Belajar Dom</h1>
12      <p class="deskripsi">paragraf kedua.</p>
13      <ul id="list-buah">
14        <li>Apel</li>
15        <li>Mangga</li>
16        <li class="favorit">durian</li>
17      </ul>
18      <button id="btn-klik">klik aku</button>
19    </div>
20    <script src="script.js"></script>
21  </body>
22  <div id="app">
23    <h1 class="judul">Belajar DOM</h1>
24    <p class="deskripsi">Paragraf pertama.</p>
25    <p class="deskripsi">Paragraf kedua.</p>
26    <ul id="list-buah">
27      <li>Apel</li>
28      <li>Mangga</li>
29      <li class="favorit">Durian</li>
30    </ul>
31    <button id="btn-klik">Klik Aku</button>
32  </div>
33  <script src="script.js"></script>
34 </body>
35 </body>
36 </html>
37
```

Buka file script.js dan isikan code berikut.jalankan dan amati hasilnya

```
1  const judul = document.querySelector('.judul')
2  const btn = document.querySelector('#btn-klik')
3
4  const paragraf = document.querySelectorAll('.deskripsi')
5  const semuaLi = document.querySelectorAll('li')
6
7  const list = document.getElementById('list-buah')
8
9  console.log('Judul:', judul)
10 console.log('Jumlah paragraf:', paragraf.length)
11 console.log('List node:', list)
12
13 const favorit = document.querySelector('.favorit')
14 console.log('Parent:', favorit.parentElement.id)
15 console.log('Sibling sebelum:', favorit.previousElementSibling?.textContent)
```

Output :

Belajar DOM

Paragraf pertama.

Paragraf kedua.

- Apel
- Mangga
- Durian

Klik Aku

C. Bagian-3 manipulasi DOM

Selain mengubah elemen yang sudah ada, kita juga bisa menciptakan elemen baru dan menambahkannya ke DOM:

- `document.createElement('tag')` — buat elemen baru (belum ada di halaman)
- `parent.appendChild(child)` — tambahkan anak ke akhir parent
- `parent.insertBefore(newNode, refNode)` — sisipkan sebelum elemen referensi
- `parent.prepend(child)` — tambahkan ke awal parent (modern)
- `parent.append(child)` — tambahkan ke akhir parent (modern, bisa terima string juga)
- `parent.insertAdjacentHTML('beforeend', htmlString)` — cara cepat sisipkan HTML string
- `elemen.remove()` — hapus elemen dari DOM
- `parent.removeChild(child)` — hapus anak dari parent (cara lama)
- `elemen.cloneNode(true)` — duplikasi elemen beserta semua isinya
- `insertAdjacentHTML` sangat berguna untuk menambahkan HTML kompleks:
 - `'beforebegin'` — sebelum elemen itu sendiri
 - `'afterbegin'` — di dalam elemen, sebelum child pertama
 - `'beforeend'` — di dalam elemen, setelah child terakhir (paling sering dipakai)
 - `'afterend'` — setelah elemen itu sendiri

Klik file manipulasi.html masukkan code berikut.

```
1  <!DOCTYPE html>
2  <html lang="id">
3
4  <head>
5      <meta charset="UTF-8">
6      <title>Manipulasi DOM</title>
7      <style>
8          body {
9              font-family: Arial;
10             padding: 20px;
11         }
12
13         .aktif {
14             background: #D5F5E3;
15             border: 2px solid #27AE60;
16             padding: 8px;
17             border-radius: 4px;
18         }
19
20         .error {
21             color: #E74C3C;
22         }
23
24         .tersembunyi {
25             display: none;
26         }
27     </style>
28 </head>
29
30 <body>
31     <h1 id="judul"> Judul Asli</h1>
32     <p id="info"> Info teks.</p>
33     
34     <div id="kotak">
35         Kotak ini akan berubah.
36     </div>
37     <button id="btn-ubah"> Ubah Semua</button>
38     <button id="btn-toggle"> Toggle Aktif</button>
39     <script src="manipulasi.js"></script>
40 </body>
41
42 </html>
```

Selanjutnya buat file manipulasi.js lalu isikan code berikut.

```
1  const judul = document.getElementById('judul')
2  const info = document.getElementById('info')
3  const foto = document.getElementById('foto')
4  const kotak = document.getElementById('kotak')
5  const btnUbah = document.getElementById('btn-ubah')
6  const btnToggle = document.getElementById('btn-toggle')
7  btnUbah.addEventListener('click', () => {
8      judul.textContent = 'Judul Sudah Diubah!'
9  })
10  info.innerHTML = 'Teks <strong>tebal</strong> dan <em>miring</em>.'
11
12  foto.setAttribute('src', 'https://picsum.photos/100/100?random=2')
13  foto.setAttribute('alt', 'Foto baru')
14  kotak.style.backgroundColor = '#FEF9E7'
15  kotak.style.padding = '12px'
16  kotak.style.borderRadius = '8px'
17  })
18  btnToggle.addEventListener('click', () => {
19      kotak.classList.toggle('aktif')
20      const adaAktif = kotak.classList.contains('aktif')
21      btnToggle.textContent = adaAktif ? 'Nonaktifkan' : 'Aktifkan'
22  })
```

Lalu setelah itu buka Kembali file manipulasi.html di browser beserta dengan devtools.dan amati di browser untuk perubahannya.

Klik Ubah Semua — judul, teks, gambar, dan kotak semuanya berubah sekaligus.

Klik Toggle Aktif

berulang kali — class aktif bergantian ditambah dan dihapus. Teks tombol ikut berubah. Buka

DevTools → Elements, klik elemen #kotak — perhatikan attribute class berubah setiap klik di panel HTML.

D. Bagian-4-events-listener

Event adalah segala sesuatu yang terjadi di halaman — klik, ketik, scroll, resize, dll.addEventListener(tipeEvent, callback) — cara modern mendaftarkan event handler. Keunggulan vs onclick di HTML: bisa daftar banyak listener untuk event yang sama, bisa dilepas dengan removeEventListener.

Event Object — otomatis dikirim ke callback, berisi informasi tentang event:

- e.target — elemen yang memicu event (yang diklik, diketik, dll.)
- e.currentTarget — elemen yang terpasang event listener-nya
- e.type — tipe event ('click', 'input', dll.)
- e.key — tombol yang ditekan (untuk keyboard event)
- e.clientX / e.clientY — posisi mouse
- e.preventDefault() — batalkan aksi default browser (misal: mencegah form submit reload halaman)
- e.stopPropagation() — hentikan event dari bubble up ke parent

Event Bubbling — event 'naik' dari target ke atas (ke parent, grandparent, dst.) sampai document.

Ini memungkinkan teknik Event Delegation: pasang satu listener di parent untuk menangani event semua anak. Sangat berguna untuk elemen yang ditambahkan secara dinamis!

Event yang sering digunakan:

- click — klik mouse | dblclick — klik ganda
- input — setiap perubahan nilai input (real-time)
- change — setelah nilai input berubah dan blur
- submit — form dikirim
- keydown / keyup / keypress — keyboard
- focus / blur — elemen mendapat/kehilangan fokus
- mouseover / mouseout — hover mouse
- scroll — halaman di-scroll
- resize — ukuran window berubah
- DOMContentLoaded — DOM siap (lebih cepat dari load)

Buat file evenst.html dan isi dengan code berikut.

```
1 <!doctype html>
2 <html lang="id">
3
4 <head>
5   <meta charset="UTF-8" />
6   <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7   <title>Demo Events JavaScript</title>
8   <style>
9     body {
10       font-family: Arial, sans-serif;
11       margin: 20px;
12       line-height: 1.6;
13     }
14
15     h2 {
16       margin-top: 40px;
17     }
18
19     #kontainer-warna {
20       display: flex;
21       gap: 10px;
22     }
23
24     .btn-warna {
25       padding: 10px 15px;
26       border: none;
27       cursor: pointer;
28       color: white;
29       border-radius: 5px;
30     }
31
32     .aktif {
33       outline: 3px solid black;
34     }
35
36     #preview-warna {
37       margin-top: 10px;
38       padding: 20px;
39       background: #eee;
40     }
41
42     textarea {
43       width: 100%;
44       height: 80px;
45     }
46
47     #hitung-char {
48       font-size: 14px;
49     }
50
51     .field {
52       margin-bottom: 10px;
53       padding: 5px;
54       border: 1px solid #ccc;
55     }
56
57     .field-aktif {
58       border-color: blue;
59       background: #eef;
60     }
61
62     #scroll-progress {
63       position: fixed;
64       top: 0;
65       left: 0;
66       height: 5px;
67       background: green;
68       width: 0%;
69     }
70
71     .spacer {
72       height: 1200px;
73     }
74   </style>
75 </head>
76
77 <body>
78   <div id="scroll-progress"></div>
79   <h1>Demo Event JavaScript</h1>
80   <!-- Event Delegation -->
81   <h2>Pilih Warna</h2>
82   <div id="kontainer-warna">
83     <button class="btn-warna" data-warna="#E74C3C" style="background: #e74c3c">
84       Merah
85     </button>
86     <button class="btn-warna" data-warna="#3498db" style="background: #3498db">
87       Biru
88     </button>
89     <button class="btn-warna" data-warna="#2ecc71" style="background: #2ecc71">
90       Hijau
91     </button>
92   </div>
93   <div id="preview-warna">Warna: </div>
94   <!-- Input Event -->
95   <input type="text" />
96   <div id="teks-area" placeholder="ketik sesuatu..."></div>
97   <div id="hitung-char">0 / 150</div>
98   <!-- Keyboard Event -->
99   <div id="log-keyboard"></div>
100  <!-- Focus & Blur -->
101  <div id="focus-blur"></div>
102  <div class="field">
103    <input type="text" class="field-fokus" placeholder="Nama" />
104  </div>
105  <div class="field">
106    <input type="text" class="field-fokus" placeholder="Email" />
107  </div>
108  <!-- Spacer for scroll -->
109  <div class="spacer"></div>
110  <script src="events.js"></script>
111 </body>
112 </html>
```

Setelah mengisi code di file events.html buka file evenst.js dan ketikkan code berikut.

```
1 const kontainerWarna = document.getElementById('kontainer-warna')
2 const preview = document.getElementById('preview-warna')
3 kontainerWarna.addEventListener('click', (e) => {
4
5     const tombol = e.target.closest('.btn-warna')
6     if (!tombol) return
7     const warna = tombol.dataset.warna
8     preview.style.backgroundColor = warna
9     preview.textContent = `Warna: ${warna}`
10
11     document.querySelectorAll('.btn-warna').forEach(b =>
12         b.classList.remove('aktif'))
13     tombol.classList.add('aktif')
14 })
15
16 const textarea = document.getElementById('teks-area')
17 const hitungChar = document.getElementById('hitung-char')
18 const MAKS = 150
19 textarea.addEventListener('input', (e) => {
20     const panjang = e.target.value.length
21     hitungChar.textContent = `${panjang} / ${MAKS}`
22     hitungChar.style.color = panjang > MAKS * 0.9 ? '#E74C3C' : '#888'
23     if (panjang > MAKS) e.target.value = e.target.value.slice(0, MAKS)
24 })
25
26 document.addEventListener('keydown', (e) => {
27     const log = document.getElementById('log-keyboard')
28
29     if (e.ctrlKey && e.key === 's') {
30         e.preventDefault()
31         log.textContent = 'Dokumen disimpan! (Ctrl+S dicegah)'
32         setTimeout(() => log.textContent = '', 2000)
33         return
34     }
35     log.textContent = `Tombol ditekan: ${e.key} | Ctrl: ${e.ctrlKey} | Shift: ${e.shiftKey}`
36 })
37 document.querySelectorAll('.field-fokus').forEach(input => {
38     input.addEventListener('focus', (e) => {
39         e.target.parentElement.classList.add('field-aktif')
40     })
41     input.addEventListener('blur', (e) => {
42         e.target.parentElement.classList.remove('field-aktif')
43     })
44 })
45
46 const progressBar = document.getElementById('scroll-progress')
47 window.addEventListener('scroll', () => {
48     const scrollMax = document.documentElement.scrollHeight - window.innerHeight
49     const persen = (window.scrollY / scrollMax) * 100
50     progressBar.style.width = `${persen}%`
51 })
```

Jalankan file events.html di browser amati hasilnya

Event Delegation: satu listener di #kontainer-warna menangani klik semua tombol warna — bahkan tombol yang ditambahkan belakangan sekalipun. .closest('.btn-warna') memastikan kita mendapat tombol meski yang diklik adalah elemen di dalamnya. e.target.dataset.warna

mengambil nilai dari attribute data-warna='#E74C3C' di HTML. data-* attribute

adalah cara standar menyimpan data di elemen. `e.preventDefault()` pada `Ctrl+S` mencegah browser menampilkan dialog 'Save Page'. Sangat penting untuk form submit juga. Scroll progress bar adalah pola umum di website blog — scroll down untuk lihat bar berjalan.

Output :

Demo Event JavaScript

Pilih Warna

Merah

Biru

Hijau

Warna: #2ECC71

Input Text

Ketik sesuatu...

0 / 150

Keyboard Event

Focus & Blur

Nama

Email

E. Bagian 5 form validasi

Buka file form-validasi.html ketikkan code berikut

```
1 <!doctype html>
2 <html lang="id">
3 <head>
4   <meta charset="UTF-8" />
5   <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6   <title>Form Validasi</title>
7   <link rel="stylesheet" href="form-validasi.css" />
8 </head>
9 <body>
10  <div class="container">
11    <h1>Daftar Akun</h1>
12    <form id="form-registrasi" novalidate>
13      <div class="field-grup" id="grup-nama">
14        <label for="nama">Nama Lengkap</label>
15        <input
16          type="text"
17          id="nama"
18          name="nama"
19          placeholder="Minimal 3 karakter"
20        />
21        <span class="pesan-error" id="error-nama"></span>
22      </div>
23      <div class="field-grup" id="grup-email">
24        <label for="email">Email</label>
25        <input
26          type="email"
27          id="email"
28          name="email"
29          placeholder="contoh@email.com"
30        />
31        <span class="pesan-error" id="error-email"></span>
32      </div>
33      <div class="field-grup" id="grup-password">
34        <label for="password">Password</label>
35        <input
36          type="password"
37          id="password"
38          name="password"
39          placeholder="Minimal 8 karakter"
40        />
41        <span class="pesan-error" id="error-password"></span>
42        <div class="indikator-kekuatan">
43          <div class="bar" id="bar-kekuatan"></div>
44        </div>
45        <span class="teks-kekuatan" id="teks-kekuatan"></span>
46      </div>
47      <div class="field-grup" id="grup-konfirmasi">
48        <label for="konfirmasi">Konfirmasi Password</label>
49        <input
50          type="password"
51          id="konfirmasi"
52          name="konfirmasi"
53          placeholder="Ulangi password"
54        />
55        <span class="pesan-error" id="error-konfirmasi"></span>
56      </div>
57      <div class="field-grup" id="grup-telp">
58        <label for="telp">Nomor Telepon</label>
59        <input type="tel" id="telp" name="telp" placeholder="08xxxxxxxxx" />
60        <span class="pesan-error" id="error-telp"></span>
61      </div>
62      <div class="field-grup">
63        <label class="label-checkbox">
64          <input type="checkbox" id="setuju" name="setuju" />
65          Saya setuju dengan <a href="#">syarat & ketentuan</a>
66        </label>
67        <span class="pesan-error" id="error-setuju"></span>
68      </div>
69      <button type="submit" id="btn-daftar">Buat Akun</button>
70    </form>
71    <div id="sukses" class="sukses tersembunyi">
72      Akun berhasil dibuat! Selamat datang.
73    </div>
74  </div>
75  <script src="form-validasi.js"></script>
76 </body>
77 </html>
78
```

Masuk ke file form-validasi.css ketikkan css berikut.

```
1 *,
2 *::before,
3 *::after {
4     box-sizing: border-box;
5     margin: 0;
6     padding: 0;
7 }
8
9 body {
10     font-family: 'Segoe UI', Arial, sans-serif;
11     background: #F0F4F8;
12     min-height: 100vh;
13     display: flex;
14     align-items: center;
15     justify-content: center;
16 }
17
18 .container {
19     background: white;
20     padding: 2rem;
21     border-radius: 12px;
22     box-shadow: 0 4px 20px rgba(0, 0, 0, 0.1);
23     width: 100%;
24     max-width: 440px;
25 }
26
27 h1 {
28     font-size: 1.5rem;
29     color: #1a1a2e;
30     margin-bottom: 1.5rem;
31     text-align: center;
32 }
33
34 .field-grup {
35     margin-bottom: 1rem;
36 }
37
38 label {
39     display: block;
40     font-size: 0.875rem;
41     font-weight: 600;
42     color: #374151;
43     margin-bottom: 0.4rem;
44 }
45
46 input[type=text],
47 input[type=email],
48 input[type=password],
49 input[type=tel] {
50     width: 100%;
51     padding: 0.6rem 0.875rem;
52     border: 2px solid #E5E7EB;
53     border-radius: 8px;
54     font-size: 0.9rem;
55     transition: border-color 0.2s, box-shadow 0.2s;
56     outline: none;
57 }
58
59 input:focus {
60     border-color: #3B82F6;
61     box-shadow: 0 0 3px rgba(59, 130, 246, 0.15);
62 }
63
64 input.valid {
65     border-color: #10B981;
66 }
67
68 input.invalid {
69     border-color: #EF4444;
70 }
71
```

```

72 .pesan-error {
73     display: block;
74     font-size: 0.78rem;
75     color: #EF4444;
76     margin-top: 0.3rem;
77     min-height: 1.1em;
78 }
79
80 .indikator-kekuatan {
81     height: 4px;
82     background: #E5E7EB;
83     border-radius: 2px;
84     margin-top: 0.5rem;
85     overflow: hidden;
86 }
87
88 .bar {
89     height: 100%;
90     width: 0;
91     border-radius: 2px;
92     transition: width 0.3s, background 0.3s;
93 }
94
95 .teks-kekuatan {
96     font-size: 0.75rem;
97     color: #888;
98     display: block;
99     margin-top: 0.25rem;
100 }
101
102 .label-checkbox {
103     display: flex;
104     align-items: center;
105     gap: 0.5rem;
106     font-weight: normal;
107     cursor: pointer;
108     font-size: 0.875rem;
109 }
110

```

```

111 #btn-daftar {
112     padding: 0.75rem;
113     background: #3B82F6;
114     color: white;
115     border: none;
116     border-radius: 8px;
117     font-size: 1rem;
118     font-weight: 600;
119     cursor: pointer;
120     margin-top: 0.5rem;
121     transition: background 0.2s;
122 }
123
124 #btn-daftar:hover {
125     background: #2563EB;
126 }
127
128 #btn-daftar:disabled {
129     background: #93C5FD;
130     cursor: not-allowed;
131 }
132
133 .sukses {
134     background: #ECFDF5;
135     border: 1px solid #10B981;
136     border-radius: 8px;
137     padding: 1rem;
138     text-align: center;
139     color: #065F46;
140     font-weight: 600;
141     margin-top: 1rem;
142 }
143
144 .tersembunyi {
145     display: none;
146 }
147

```

selanjutnya buka file form-validasi.js ketikkan code berikut.

```
1 const form = document.getElementById('form-registrasi')
2 const inputNama = document.getElementById('nama')
3 const inputEmail = document.getElementById('email')
4 const inputPass = document.getElementById('password')
5 const inputKonf = document.getElementById('konfirmasi')
6 const inputTelp = document.getElementById('telp')
7 const inputSetuju = document.getElementById('setuju')
8 const sukses = document.getElementById('sukses')
9 // — Helper: tampilkan / sembunyikan error —————
10 function setError(inputEl, errorId, pesan) {
11   const errorEl = document.getElementById(errorId)
12   errorEl.textContent = pesan
13   inputEl.classList.toggle('invalid', pesan !== '')
14   inputEl.classList.toggle('valid', pesan === '')
15   return pesan === '' // true jika valid
16 }
17 function validasiNama(nilai) {
18   if (!nilai.trim()) return 'Nama tidak boleh kosong'
19   if (nilai.trim().length < 3) return 'Nama minimal 3 karakter'
20   if (nilai.trim().length > 50) return 'Nama maksimal 50 karakter'
21   return '' // kosong = valid
22 }
23 function validasiEmail(nilai) {
24   if (!nilai) return 'Email tidak boleh kosong'
25   const pola = /^[^\s@]+@[^\s@]+\.[^\s@]+$/
26   if (!pola.test(nilai)) return 'Format email tidak valid'
27   return ''
28 }
29 function validasiPassword(nilai) {
30   if (!nilai) return 'Password tidak boleh kosong'
31   if (nilai.length < 8) return 'Password minimal 8 karakter'
32   if (!/[A-Z]/.test(nilai)) return 'Harus ada minimal 1 huruf kapital'
33   if (!/[0-9]/.test(nilai)) return 'Harus ada minimal 1 angka'
34   return ''
35 }
36 function hitungKekuatan(nilai) {
37   let skor = 0
38   if (nilai.length >= 8) skor++
39   if (nilai.length >= 12) skor++
40   if (/[A-Z]/.test(nilai)) skor++
41   if (/[0-9]/.test(nilai)) skor++
42   if (/[A-Za-z0-9]/.test(nilai)) skor++ // karakter spesial
43   return skor
44 }
45 function validasiKonfirmasi(nilai) {
46   if (!nilai) return 'Konfirmasi password tidak boleh kosong'
47   if (nilai !== inputPass.value) return 'Password tidak cocok'
48   return ''
49 }
50 function validasiTelp(nilai) {
51   if (!nilai) return 'Nomor telepon tidak boleh kosong'
52   const pola = /^(+62|0)[0-9]{9,12}$/
53   if (!pola.test(nilai.replace(/\s|~/g, ''))) return 'Format: 08xxxxxxxx (10-13 digit)'
54   return ''
55 }
56 // — Event: validasi real-time saat blur —————
57 inputNama.addEventListener('blur', () => setError(inputNama, 'error-nama',
58   validasiNama(inputNama.value)))
59
60 inputEmail.addEventListener('blur', () => setError(inputEmail, 'error-email',
61   validasiEmail(inputEmail.value)))
62 inputPass.addEventListener('blur', () => setError(inputPass, 'error-password',
63   validasiPassword(inputPass.value)))
64 inputKonf.addEventListener('blur', () => setError(inputKonf, 'error-konfirmasi',
65   validasiKonfirmasi(inputKonf.value)))
66 inputTelp.addEventListener('blur', () => setError(inputTelp, 'error-telp',
67   validasiTelp(inputTelp.value)))
68 // — Password strength indicator —————
69 inputPass.addEventListener('input', () => {
70   const skor = hitungKekuatan(inputPass.value)
71   const bar = document.getElementById('bar-kekuatan')
72   const teks = document.getElementById('teks-kekuatan')
73   const level = ['', 'Sangat Lemah', 'Lemah', 'Sedang', 'Kuat', 'Sangat Kuat']
74   const warna = ['', '#F44444', '#F97316', '#EAB308', '#22C55E', '#16A34A']
75   bar.style.width = `${(skor / 5) * 100}%`
76   bar.style.background = warna[skor] || ''
77   teks.textContent = inputPass.value ? level[skor] || '' : ''
78 })
```

Lanjutan dari form-validasi.js

```
1 form.addEventListener('submit', (e) => {
2   e.preventDefault() // cegah reload halaman!
3   const valid = [
4     setError(inputNama, 'error-nama', validasiNama(inputNama.value)),
5     setError(inputEmail, 'error-email', validasiEmail(inputEmail.value)),
6     setError(inputPass, 'error-password', validasiPassword(inputPass.value)),
7     setError(inputKonf, 'error-konfirmasi', validasiKonfirmasi(inputKonf.value)),
8     setError(inputTelp, 'error-telp', validasiTelp(inputTelp.value)),
9   ]
10  if (!inputSetuju.checked) {
11    document.getElementById('error-setuju').textContent =
12      'Kamu harus menyetujui syarat & ketentuan'
13    valid.push(false)
14  } else {
15    document.getElementById('error-setuju').textContent = ''
16  }
17  if (valid.every(v => v)) {
18    // Semua valid — kirim data
19    sukses.classList.remove('tersembunyi')
20    form.reset()
21    // Reset semua class valid/invalid
22    form.querySelectorAll('input').forEach(i => {
23      i.classList.remove('valid', 'invalid')
24    })
25  } else {
26    // Fokus ke field pertama yang error
27    const firstInvalid = form.querySelector('.invalid')
28    firstInvalid?.focus()
29    firstInvalid?.scrollIntoView({ behavior: 'smooth', block: 'center' })
30  }
31 }
32 })
```

Buka file form-validasi.html di browser, uji semua skenario validasi dan amati hasilnya.

Amati Hasilnya

Border merah/hijau pada input muncul saat blur — memberikan feedback visual yang jelas tanpa mengganggu saat user masih mengetik. Password strength bar bergerak dan berubah warna secara real-time saat mengetik — coba password lemah vs kuat. Klik Buat Akun tanpa isi apapun — semua error muncul sekaligus, dan fokus berpindah ke field pertama yang invalid. `e.preventDefault()` pada submit sangat penting — tanpanya, browser akan reload halaman dan semua data hilang. Regex `^[^\s@]+@[^\s@]+\.[^\s@]+$` — validasi email sederhana. Untuk production, validasi email sebaiknya dilakukan di server juga.

F. Latihan -1.Html Kalkulator Interaktif

```
1 <!doctype html>
2 <html lang="id">
3 <head>
4   <meta charset="UTF-8" />
5   <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6   <title>Kalkulator Interaktif</title>
7
8   <style>
9     body {
10       font-family: Arial, sans-serif;
11       background: #f3f4f6;
12       display: flex;
13       justify-content: center;
14       align-items: center;
15       height: 100vh;
16     }
17
18     .calculator {
19       background: white;
20       padding: 20px;
21       border-radius: 12px;
22       box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
23       width: 280px;
24     }
25
26     .display {
27       width: 100%;
28       height: 60px;
29       margin-bottom: 15px;
30       font-size: 24px;
31       text-align: right;
32       padding: 10px;
33       border: 2px solid #ddd;
34       border-radius: 8px;
35       box-sizing: border-box;
36     }
37
38     .buttons {
39       display: grid;
40       grid-template-columns: repeat(4, 1fr);
41       gap: 10px;
42     }
43
44     button {
45       padding: 15px;
46       font-size: 18px;
47       border: none;
48       border-radius: 8px;
49       cursor: pointer;
50       background: #e5e7eb;
51       transition: 0.2s;
52     }
53
54     button:hover {
55       background: #d1d5db;
56     }
57
58     .operator {
59       background: #60a5fa;
60       color: white;
61     }
62
63     .equal {
64       background: #22c55e;
65       color: white;
66     }
67
68     .clear {
69       background: #ef4444;
70       color: white;
71     }
72   </style>
73 </head>
74
```

Lanjutan bagian dari latihan-1.html

```
1  <body>
2    <div class="calculator">
3      <input type="text" class="display" id="display" readonly />
4
5      <div class="buttons" id="buttons">
6        <button data-value="7">7</button>
7        <button data-value="8">8</button>
8        <button data-value="9">9</button>
9        <button class="operator" data-value="/">÷</button>
10
11        <button data-value="4">4</button>
12        <button data-value="5">5</button>
13        <button data-value="6">6</button>
14        <button class="operator" data-value="*">×</button>
15
16        <button data-value="1">1</button>
17        <button data-value="2">2</button>
18        <button data-value="3">3</button>
19        <button class="operator" data-value="-">-</button>
20
21        <button data-value="0">0</button>
22        <button data-value=".">.</button>
23        <button class="equal" data-value="=">=</button>
24        <button class="operator" data-value="+">+</button>
25
26        <button class="clear" data-value="C" style="grid-column: span 4">
27          C
28        </button>
29      </div>
30    </div>
31
32    <script src="latihan-1.js"></script>
33  </body>
34 </html>
```

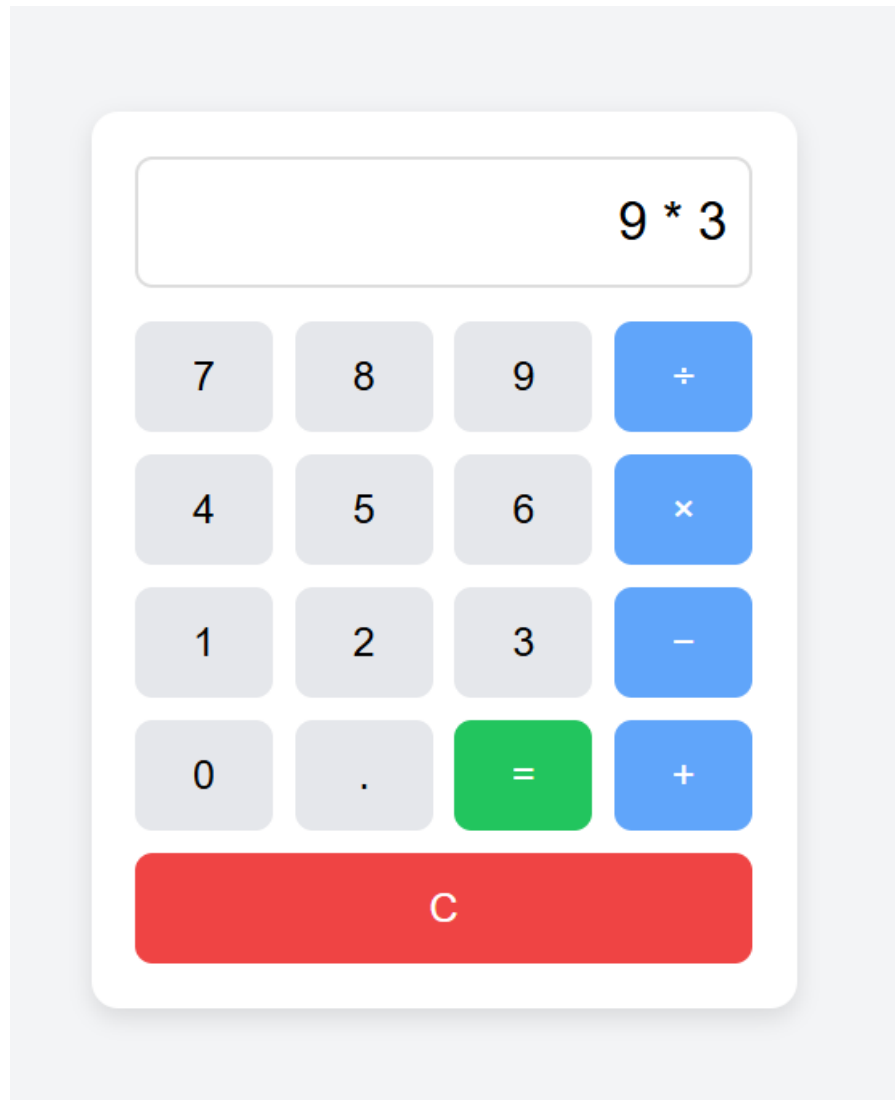
Latihan-1.js

```
1  const display = document.getElementById("display");
2  const buttons = document.getElementById("buttons");
3
4  let firstNumber = "";
5  let secondNumber = "";
6  let operator = "";
7  let isSecondNumber = false;
8
9  buttons.addEventListener("click", function (e) {
10     const value = e.target.dataset.value;
11
12     if (!value) return;
13
14     handleInput(value);
15 });
16
17 // HANDLE INPUT
18 function handleInput(value) {
19
20     // CLEAR
21     if (value === "C") {
22         clearCalculator();
23         return;
24     }
25
26     // EQUAL
27     if (value === "=") {
28         calculate();
29         return;
30     }
31
32     // OPERATOR
33     if (["+","-", "*", "/"].includes(value)) {
34
35         if (firstNumber !== "") {
36             operator = value;
37             isSecondNumber = true;
38             display.value = `${firstNumber} ${operator}`;
39         }
40
41         return;
42     }
43
44     // ANGKA / DESIMAL
45     if (!isSecondNumber) {
46         firstNumber += value;
47         display.value = firstNumber;
48     } else {
49         secondNumber += value;
50         display.value = `${firstNumber} ${operator} ${secondNumber}`;
51     }
52 }
53
```

Lanjutan bagian dari Lathan-1.js

```
1  function calculate() {
2
3      if (firstNumber === "" || secondNumber === "") return;
4
5      let result;
6
7      const num1 = parseFloat(firstNumber);
8      const num2 = parseFloat(secondNumber);
9
10     switch (operator) {
11         case "+":
12             result = num1 + num2;
13             break;
14
15         case "-":
16             result = num1 - num2;
17             break;
18
19         case "*":
20             result = num1 * num2;
21             break;
22
23         case "/":
24             result = num1 / num2;
25             break;
26
27         default:
28             return;
29     }
30
31     display.value = result;
32
33     // reset untuk perhitungan berikutnya
34     firstNumber = result.toString();
35     secondNumber = "";
36     operator = "";
37     isSecondNumber = false;
38 }
39
40 // CLEAR
41 function clearCalculator() {
42     display.value = "";
43     firstNumber = "";
44     secondNumber = "";
45     operator = "";
46     isSecondNumber = false;
47 }
48
49 // KEYBOARD SUPPORT
50 document.addEventListener("keydown", function (e) {
51
52     const key = e.key;
53
54     // angka dan operator
55     if (
56         /[0-9]/.test(key) ||
57         ["+", "-", "*", "/", "."].includes(key)
58     ) {
59         handleInput(key);
60     }
61
62     // enter = hasil
63     else if (key === "Enter") {
64         calculate();
65     }
66
67     // escape = clear
68     else if (key === "Escape") {
69         clearCalculator();
70     }
71 });
```

Output :



G. Latihan-2.Html Daftar Produk

```
1 <!doctype html>
2 <html lang="id">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <title>Shopping Cart</title>
7
8     <style>
9       * {
10         margin: 0;
11         padding: 0;
12         box-sizing: border-box;
13         font-family: Arial, sans-serif;
14       }
15
16       body {
17         background: #f3f4f6;
18         padding: 20px;
19       }
20
21       h1 {
22         margin-bottom: 20px;
23         text-align: center;
24       }
25
26       .container {
27         display: grid;
28         grid-template-columns: 2fr 1fr;
29         gap: 20px;
30       }
31
32       /* PRODUK */
33       .products {
34         display: grid;
35         grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
36         gap: 20px;
37       }
38
39       .card {
40         background: white;
41         border-radius: 12px;
42         overflow: hidden;
43         box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
44       }
45
46       .card img {
47         width: 100%;
48         height: 180px;
49         object-fit: cover;
50       }
51
52       .card-content {
53         padding: 15px;
54       }
55
56       .card h3 {
57         margin-bottom: 10px;
58       }
59
60       .price {
61         color: green;
62         font-weight: bold;
63         margin-bottom: 12px;
64       }
65
66       .btn {
67         width: 100%;
68         padding: 10px;
69         border: none;
70         background: #2563eb;
71         color: white;
72         border-radius: 8px;
73         cursor: pointer;
74       }
```

Lanjutan Latihan-2.html

```
1  .btn:hover {
2    background: #1d4ed8;
3  }
4
5  /* KERANJANG */
6  .cart {
7    background: white;
8    padding: 20px;
9    border-radius: 12px;
10   box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
11   height: fit-content;
12   position: sticky;
13   top: 20px;
14 }
15
16 .cart-header {
17   display: flex;
18   justify-content: space-between;
19   align-items: center;
20   margin-bottom: 20px;
21 }
22
23 .badge {
24   background: red;
25   color: white;
26   padding: 4px 10px;
27   border-radius: 999px;
28   font-size: 14px;
29 }
30
31 .cart-item {
32   border-bottom: 1px solid #ddd;
33   padding: 10px 0;
34 }
35
36 .cart-item h4 {
37   margin-bottom: 5px;
38 }
39
40 .qty-control {
41   display: flex;
42   align-items: center;
43   gap: 10px;
44   margin-top: 10px;
45 }
46
47 .qty-btn {
48   width: 30px;
49   height: 30px;
50   border: none;
51   background: #e5e7eb;
52   border-radius: 6px;
53   cursor: pointer;
54 }
55
56 .remove-btn {
57   margin-top: 10px;
58   padding: 6px 10px;
59   border: none;
60   background: crimson;
61   color: white;
62   border-radius: 6px;
63   cursor: pointer;
64 }
65
66 .total {
67   margin-top: 20px;
68   font-size: 18px;
69   font-weight: bold;
70 }
71
72 .checkout {
73   width: 100%;
74   margin-top: 15px;
75   padding: 12px;
76   border: none;
77   background: green;
78   color: white;
79   border-radius: 8px;
80   cursor: pointer;
81   font-size: 16px;
82 }
83
84 .checkout:hover {
85   background: darkgreen;
86 }
87
88 @media (max-width: 768px) {
89   .container {
90     grid-template-columns: 1fr;
91   }
92 }
```

```

1   </style>
2   </head>
3
4   <body>
5     <h1>Shopping Cart</h1>
6
7     <div class="container">
8       <!-- PRODUCTS -->
9       <div class="products" id="product-list">
10        <div class="card">
11          
12          <div class="card-content">
13            <h3>Laptop</h3>
14            <p class="price">Rp 7.000.000</p>
15            <button
16              class="btn"
17              data-id="1"
18              data-name="Laptop"
19              data-price="7000000"
20            >
21              Tambah ke Keranjang
22            </button>
23          </div>
24        </div>
25
26        <div class="card">
27          
28          <div class="card-content">
29            <h3>Headphone</h3>
30            <p class="price">Rp 500.000</p>
31            <button
32              class="btn"
33              data-id="2"
34              data-name="Headphone"
35              data-price="500000"
36            >
37              Tambah ke Keranjang
38            </button>
39          </div>
40        </div>
41
42        <div class="card">
43          
44          <div class="card-content">
45            <h3>Keyboard</h3>
46            <p class="price">Rp 350.000</p>
47            <button
48              class="btn"
49              data-id="3"
50              data-name="Keyboard"
51              data-price="350000"
52            >
53              Tambah ke Keranjang
54            </button>
55          </div>
56        </div>
57
58        <div class="card">
59          
60          <div class="card-content">
61            <h3>Mouse</h3>
62            <p class="price">Rp 200.000</p>
63            <button
64              class="btn"
65              data-id="4"
66              data-name="Mouse"
67              data-price="200000"
68            >
69              Tambah ke Keranjang
70            </button>
71          </div>
72        </div>
73
74        <div class="card">
75          
76          <div class="card-content">
77            <h3>Monitor</h3>
78            <p class="price">Rp 2.000.000</p>
79            <button
80              class="btn"
81              data-id="5"
82              data-name="Monitor"
83              data-price="2000000"
84            >
85              Tambah ke Keranjang
86            </button>
87          </div>
88        </div>
89      </div>
90
91      <!-- KERANJANG -->
92      <div class="cart">
93        <div class="cart-header">
94          <h2>Keranjang</h2>
95          <span class="badge" id="cart-count">0</span>
96        </div>
97
98        <div id="cart-items"></div>
99
100       <div class="total">Total: Rp <span id="total-price">0</span></div>
101
102       <button class="checkout" id="checkout-btn">Checkout</button>
103     </div>
104   </body>
105   <script src="latihan-2.js"></script>
106 </html>

```

Latihan-2.js

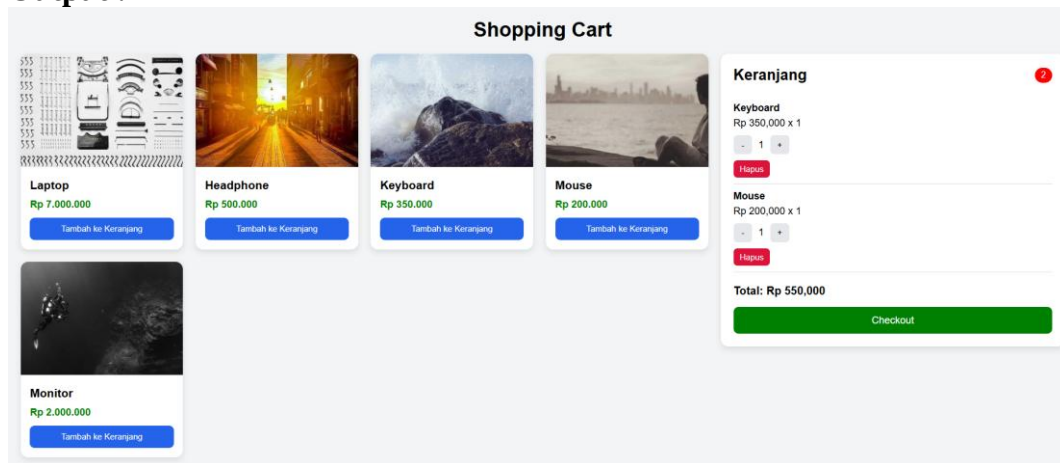
```
1  const productList = document.getElementById("product-list");
2  const cartItems = document.getElementById("cart-items");
3  const totalPrice = document.getElementById("total-price");
4  const cartCount = document.getElementById("cart-count");
5  const checkoutBtn = document.getElementById("checkout-btn");
6
7  // STATE KERANJANG
8  let cart = [];
9
10 // TAMBAH PRODUK
11 productList.addEventListener("click", function (e) {
12
13   if (e.target.classList.contains("btn")) {
14
15    const id = e.target.dataset.id;
16    const name = e.target.dataset.name;
17    const price = Number(e.target.dataset.price);
18
19    const existingItem = cart.find(item => item.id === id);
20
21    if (existingItem) {
22     existingItem.qty++;
23   } else {
24     cart.push({
25       id,
26       name,
27       price,
28       qty: 1
29     });
30   }
31
32   renderCart();
33 }
34 });
35
36 // RENDER KERANJANG
37 function renderCart() {
38
39   cartItems.innerHTML = "";
40
41   let total = 0;
42   let totalItems = 0;
43
44   cart.forEach(item => {
45
46     total += item.price * item.qty;
47     totalItems += item.qty;
48
49     cartItems.innerHTML += `
50 <div class="cart-item">
51
52   <h4>${item.name}</h4>
53
54   <p>
55     Rp ${item.price.toLocaleString()} x ${item.qty}
56   </p>
57
58   <div class="qty-control">
59
60     <button class="qty-btn minus" data-id="${item.id}">
61       -
62     </button>
63
64     <span>${item.qty}</span>
65
66     <button class="qty-btn plus" data-id="${item.id}">
67       +
68     </button>
69
70   </div>
71
72   <button class="remove-btn" data-id="${item.id}">
73     Hapus
74   </button>
75
76 </div>
77 `;
78   });
79 }
```

```

1   totalPrice.textContent = total.toLocaleString();
2   cartCount.textContent = totalItems;
3 }
4
5 // EVENT KERANJANG
6 cartItems.addEventListener("click", function (e) {
7
8     const id = e.target.dataset.id;
9
10    // TAMBAH JUMLAH
11    if (e.target.classList.contains("plus")) {
12
13        const item = cart.find(item => item.id === id);
14        item.qty++;
15
16        renderCart();
17    }
18
19    // KURANG JUMLAH
20    if (e.target.classList.contains("minus")) {
21
22        const item = cart.find(item => item.id === id);
23
24        if (item.qty > 1) {
25            item.qty--;
26        }
27
28        renderCart();
29    }
30
31    // HAPUS ITEM
32    if (e.target.classList.contains("remove-btn")) {
33
34        cart = cart.filter(item => item.id !== id);
35
36        renderCart();
37    }
38 });
39
40 // CHECKOUT
41 checkoutBtn.addEventListener("click", function () {
42
43     if (cart.length === 0) {
44         alert("Keranjang masih kosong!");
45         return;
46     }
47
48     let summary = "=== RINGKASAN ORDER ===\n\n";
49
50     cart.forEach(item => {
51         summary += `
52     ${item.name} (${item.qty}) = Rp ${(item.price * item.qty).toLocaleString()}
53     `;
54     });
55
56     summary += `\nTotal: Rp ${totalPrice.textContent}`;
57
58     alert(summary);
59 });

```

Output :



H. Tugas.html

```
1  <!doctype html>
2  <html lang="id">
3    <head>
4      <meta charset="UTF-8" />
5      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6      <title>Todo List Lengkap</title>
7      <link rel="stylesheet" href="style.css" />
8    </head>
9    <body>
10     <div class="container">
11       <h1>Todo List</h1>
12
13       <form id="todoForm">
14         <input type="text" id="taskInput" placeholder="Masukkan tugas..." />
15
16         <select id="priority">
17           <option value="Rendah">Rendah</option>
18           <option value="Sedang">Sedang</option>
19           <option value="Tinggi">Tinggi</option>
20         </select>
21
22         <button type="submit">Tambah</button>
23       </form>
24
25       <p id="error"></p>
26
27       <div class="filters">
28         <button data-filter="all">Semua</button>
29         <button data-filter="active">Aktif</button>
30         <button data-filter="completed">Selesai</button>
31       </div>
32
33       <ul id="taskList"></ul>
34
35       <div class="bottom">
36         <p id="counter">0 tugas tersisa</p>
37         <button id="clearCompleted">Hapus Semua Selesai</button>
38       </div>
39     </div>
40
41     <script src="script.js"></script>
42   </body>
43 </html>
44
```

Tugas.js

```
1 const todoForm = document.getElementById("todoForm");
2 const taskInput = document.getElementById("taskInput");
3 const priorityInput = document.getElementById("priority");
4 const taskList = document.getElementById("taskList");
5 const counter = document.getElementById("counter");
6 const error = document.getElementById("error");
7 const clearCompleted = document.getElementById("clearCompleted");
8
9 let todos = JSON.parse(localStorage.getItem("todos")) || [];
10 let currentFilter = "all";
11
12 renderTodos();
13
14 todoForm.addEventListener("submit", function (e) {
15   e.preventDefault();
16
17   const text = taskInput.value.trim();
18
19   // VALIDASI
20   if (text === "") {
21     error.textContent = "Tugas tidak boleh kosong!";
22     return;
23   }
24   if (text.length < 3) {
25     error.textContent = "Minimal 3 karakter!";
26     return;
27   }
28   if (text.length > 100) {
29     error.textContent = "Maksimal 100 karakter!";
30     return;
31   }
32   error.textContent = "";
33
34   const todo = {
35     id: Date.now(),
36     text,
37     completed: false,
38     priority: priorityInput.value,
39   };
40   todos.push(todo);
41   saveTodos();
42   renderTodos();
43   taskInput.value = "";
44 });
45
46 function renderTodos() {
47   taskList.innerHTML = "";
48
49   let filteredTodos = todos.filter((todo) => {
50     if (currentFilter === "active") return !todo.completed;
51     if (currentFilter === "completed") return todo.completed;
52     return true;
53   });
54
55   filteredTodos.forEach((todo) => {
56     const li = document.createElement("li");
57     li.draggable = true;
58
59     if (todo.completed) {
60       li.classList.add("completed");
61     }
62     // DRAG START
63     li.addEventListener("dragstart", () => {
64       li.classList.add("dragging");
65     });
66     // DRAG END
67     li.addEventListener("dragend", () => {
68       li.classList.remove("dragging");
69       updateDraggedOrder();
70     });
71
72     li.innerHTML = `
73     <div class="task-info">
74       <input type="checkbox" ${todo.completed ? "checked" : ""}>
75       <span class="task-text">${todo.text}</span>
76       <span class="priority">${todo.priority}</span>
77     </div>
78     <button class="delete-btn">Hapus</button>
79   `;
80
81     const checkbox = li.querySelector("input");
82     const deleteBtn = li.querySelector(".delete-btn");
83     const taskText = li.querySelector(".task-text");
84
85     // CHECKBOX
86     checkbox.addEventListener("change", () => {
87       todo.completed = checkbox.checked;
88       saveTodos();
89       renderTodos();
90     });
91
92     // HAPUS
93     deleteBtn.addEventListener("click", () => {
94       todos = todos.filter((t) => t.id !== todo.id);
95       saveTodos();
96       renderTodos();
97     });
98
99     // EDIT
100     taskText.addEventListener("dblclick", () => {
101       const inputEdit = document.createElement("input");
102       inputEdit.type = "text";
103       inputEdit.value = todo.text;
104
105       taskText.replaceWith(inputEdit);
106       inputEdit.focus();
107
108       function saveEdit() {
109         if (inputEdit.value.trim() !== "") {
110           todo.text = inputEdit.value.trim();
111           saveTodos();
112           renderTodos();
113         }
114       }
115       inputEdit.addEventListener("blur", saveEdit);
116       inputEdit.addEventListener("keypress", (e) => {
117         if (e.key === "Enter") {
118           saveEdit();
119         }
120       });
121     });
122     taskList.appendChild(li);
123   });
124   updateCounter();
125 }
```

Lanjutan Dari File Tugas.Js

```
1 function updateCounter() {
2   const activeTasks = todos.filter((todo) => !todo.completed).length;
3   counter.textContent = `${activeTasks} tugas tersisa`;
4 }
5
6 function saveTodos() {
7   localStorage.setItem("todos", JSON.stringify(todos));
8 }
9
10 // FILTER
11 document.querySelectorAll(".filters button").forEach((button) => {
12   button.addEventListener("click", () => {
13     currentFilter = button.dataset.filter;
14     renderTodos();
15   });
16 });
17 clearCompleted.addEventListener("click", () => {
18   todos = todos.filter((todo) => !todo.completed);
19   saveTodos();
20   renderTodos();
21 });
22 // DRAG & DROP
23 taskList.addEventListener("dragover", (e) => {
24   e.preventDefault();
25
26   const dragging = document.querySelector(".dragging");
27   const afterElement = getDragAfterElement(taskList, e.clientY);
28
29   if (afterElement == null) {
30     taskList.appendChild(dragging);
31   } else {
32     taskList.insertBefore(dragging, afterElement);
33   }
34 });
35
36 function getDragAfterElement(container, y) {
37   const elements = [...container.querySelectorAll("li:not(.dragging)")];
38
39   return elements.reduce(
40     (closest, child) => {
41       const box = child.getBoundingClientRect();
42
43       const offset = y - box.top - box.height / 2;
44
45       if (offset < 0 && offset > closest.offset) {
46         return { offset, element: child };
47       } else {
48         return closest;
49       }
50     },
51     { offset: Number.NEGATIVE_INFINITY },
52   ).element;
53 }
54
55 function updateDraggedOrder() {
56   const items = [...taskList.children];
57
58   const newTodos = [];
59
60   items.forEach((item) => {
61     const text = item.querySelector(".task-text").textContent;
62
63     const found = todos.find((todo) => todo.text === text);
64
65     if (found) {
66       newTodos.push(found);
67     }
68   });
69   todos = newTodos;
70   saveTodos();
71 }
72
```

Tugas.css

```
1  * {
2    margin: 0;
3    padding: 0;
4    box-sizing: border-box;
5    font-family: Arial, sans-serif;
6  }
7
8  body {
9    background: #f4f4f4;
10   display: flex;
11   justify-content: center;
12   padding: 40px;
13 }
14
15 .container {
16   width: 100%;
17   max-width: 600px;
18   background: white;
19   padding: 25px;
20   border-radius: 10px;
21   box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
22 }
23
24 h1 {
25   text-align: center;
26   margin-bottom: 20px;
27 }
28
29 form {
30   display: flex;
31   gap: 10px;
32   margin-bottom: 10px;
33 }
34
35 input,
36 select,
37 button {
38   padding: 10px;
39   border-radius: 5px;
40   border: 1px solid #ccc;
41 }
42
43 input {
44   flex: 1;
45 }
46
47 button {
48   cursor: pointer;
49   border: none;
50   background: #6a0dad;
51   color: white;
52 }
53
54 button:hover {
55   opacity: 0.9;
56 }
```

Lanjutan Dari Tugas.Css

```
1  #error {
2    color: red;
3    margin-bottom: 10px;
4  }
5
6  .filters {
7    display: flex;
8    gap: 10px;
9    margin: 15px 0;
10 }
11
12 ul {
13   list-style: none;
14 }
15
16 li {
17   background: #f9f9f9;
18   padding: 12px;
19   margin-bottom: 10px;
20   border-radius: 5px;
21   display: flex;
22   justify-content: space-between;
23   align-items: center;
24 }
25
26 .task-info {
27   display: flex;
28   align-items: center;
29   gap: 10px;
30 }
31
32 .completed span {
33   text-decoration: line-through;
34   color: gray;
35 }
36
37 .priority {
38   padding: 3px 8px;
39   border-radius: 5px;
40   color: white;
41   font-size: 12px;
42 }
43
44 .Rendah {
45   background: green;
46 }
47
48 .Sedang {
49   background: orange;
50 }
51
52 .Tinggi {
53   background: red;
54 }
55
56 .bottom {
57   display: flex;
58   justify-content: space-between;
59   align-items: center;
60   margin-top: 15px;
61 }
```

BAB IV PENUTUP

4.1 KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa JavaScript DOM (Document Object Model) merupakan antarmuka pemrograman yang memungkinkan JavaScript untuk mengakses, mengubah, dan memanipulasi elemen-elemen HTML secara dinamis tanpa perlu memuat ulang halaman web. Dalam praktikum ini, saya mempelajari cara memilih elemen menggunakan metode seperti `getElementById`, `querySelector`, dan `getElementsByClassName`, serta memanipulasi konten, atribut, dan style elemen secara langsung melalui script. Saya juga memahami konsep event handling, yaitu bagaimana JavaScript merespons interaksi pengguna seperti klik, input, dan hover untuk menciptakan halaman web yang interaktif. Dengan demikian, penguasaan JavaScript DOM menjadi langkah penting dalam pengembangan web modern, karena DOM menjadi jembatan antara logika pemrograman JavaScript dengan tampilan HTML yang dilihat pengguna. Praktikum ini memberikan dasar yang kuat dalam membangun halaman web yang dinamis, responsif, dan interaktif.

4.2 SARAN

Berdasarkan praktikum yang telah dilaksanakan, terdapat beberapa saran yang diharapkan dapat meningkatkan pemahaman dan kualitas pembelajaran di masa mendatang:

1. Perbanyak Latihan Mandiri Mahasiswa disarankan untuk lebih sering berlatih memanipulasi DOM secara mandiri agar semakin terbiasa dalam mengakses dan mengubah elemen HTML menggunakan JavaScript secara dinamis.
2. Pelajari JavaScript DOM Secara Bertahap Pembelajaran sebaiknya dilakukan secara sistematis, dimulai dari konsep dasar seperti selektor elemen dan manipulasi konten, hingga materi lanjutan seperti event handling, DOM traversal, dan pembuatan elemen secara dinamis.
3. Manfaatkan Sumber Belajar yang Terpercaya Mahasiswa dianjurkan untuk memanfaatkan referensi pembelajaran yang kredibel seperti dokumentasi resmi MDN Web Docs, W3Schools, serta video tutorial dari platform terpercaya guna memperdalam pemahaman tentang JavaScript DOM.
4. Tingkatkan Kreativitas dalam Membangun Antarmuka Interaktif Mahasiswa diharapkan mampu mengeksplorasi penerapan JavaScript DOM dalam membangun fitur-fitur interaktif yang inovatif, seperti validasi formulir, perubahan tampilan dinamis, dan manipulasi data secara real-time pada halaman web.